Parameter-Efficient Fine-Tuning of Large Language Models: A Survey

Parameter-efficient fine-tuning (PEFT) refers to a family of techniques that adapt a large pretrained language model to a new task by training a small subset of parameters, or a small learned addition to the model, while the pretrained weights themselves stay frozen. The literature usually organizes this family by mechanism: modules bolted onto the network, structured edits to individual weight matrices, extensions of the input sequence, and masks or offsets over existing weights. That split is useful for exposition, but it hides the axis that actually decides what a method costs to run in production: whether the trained delta can be folded back into the frozen backbone, or has to ride along at inference time forever. Working through adapters, the LoRA family, prompt and prefix tuning, and selective and sparse fine-tuning surfaces that hidden axis rather than papering over it, and the argument closes on two problems the field has not solved: why particular mechanisms outperform others at all, and when two independently trained adaptations can be merged without one damaging the other.

1 Introduction

Full fine-tuning of a large language model is not just slow; it is a memory problem before it is anything else. Naively fine-tuning a 65-billion-parameter model in 16-bit precision needs more than 780 gigabytes of GPU memory once gradients and the optimizer's running statistics are counted alongside the weights themselves, an order of magnitude beyond what a single high-end accelerator, or even a small cluster of them, provides (Dettmers et al., 2023). Against GPT-3′s 175 billion parameters, Hu et al. (2021) report that training a low-rank update instead of every weight cuts the number of trainable parameters by 10,000-fold and the GPU memory requirement by threefold, using the Adam optimizer as the baseline in both cases. Those two numbers are the entire case for parameter-efficient fine-tuning (PEFT): freeze the pretrained weights, and train only a small number of additional parameters, or a small selected subset of the existing ones, instead of every weight in the network.

The saving is not only a training-time convenience. A team that serves ten customer-specific or task-specific variants of one base model pays a storage cost proportional to ten copies of the trained delta rather than ten copies of the full model, provided the delta stays small, which is precisely the property each family of methods below trades off against something else: training stability, inference latency, or how the delta composes with other deltas trained the same way. The rest of this survey works through what each

family actually does to earn that saving, where the saving runs out, and which one an engineer with a specific constraint should reach for.

2 A Taxonomy of Parameter-Efficient Fine-Tuning

A trained delta has to live somewhere relative to the frozen backbone, and where it lives is the cleanest way to group the methods that follow. Four placements recur across the literature: a new module inserted into the forward pass (additive methods, exemplified by adapters), a structured edit written directly into an existing weight matrix (reparameterization methods, exemplified by LoRA), an extension of the input sequence itself (prompt-based methods), and a mask or elementwise edit layered over existing weights or activations (selective and sparse methods). Ding et al. (2022) group much of this same territory into addition-based, specification-based, and reparameterization-based tuning, and that split is not wrong so much as it is a step removed from the mechanism: it classifies methods by what they change, while this survey classifies them by where the change sits relative to the frozen computation, because that placement is what turns out to determine deployment cost later in this document.

The placement axis earns its keep on a case that resists it cleanly. $(IA)^3$ (Liu et al., 2022) learns a vector per layer that rescales the attention keys, the attention values, and the feed-forward activations elementwise. It adds no module, so it is not additive in the sense adapters are; it is not a low-rank edit to a weight matrix, so it is not reparameterization in the sense LoRA is. What it shares with the selective and sparse family is that its rescaling vector can be pre-multiplied into an adjacent weight matrix once, permanently (the same "lives in weight space" property that defines that family), so it is placed there in this survey, on that basis, rather than filed under a fifth, ad hoc category. A reader who places it under reparameterization instead, on the grounds that a diagonal rescale is itself a restricted structured edit, would not be wrong; the point of naming the criterion explicitly is that the disagreement becomes visible and arguable, rather than silently baked into which section a method happens to appear in.

The four-family split organizes the exposition that follows, and it should not be mistaken for the axis that determines what any of these methods actually costs to run. Two methods filed in the same family here can have opposite inference-time footprints, and two methods filed in different families can share the same one: what decides whether an adapted model pays a permanent cost at every future inference call is not which of these four buckets a method sits in, but whether its trained delta can be folded back into the original weight matrices once training ends, or has to remain in the forward pass (as a module, or as extra positions in the input) for the life of the deployment. That distinction cuts across the four families built here rather than along them, and working it out in full is the job of the section titled "Cross-Family Comparison," after each family has been described on its own terms.

3 Adapter Modules

A bottleneck adapter is a small feed-forward module (down-project to a low dimension, apply a nonlinearity, project back up) inserted with its own residual connection after the attention sublayer and after the feed-forward sublayer of each transformer block, everywhere the pretrained weights otherwise stay frozen (Houlsby et al., 2019). The projection matrices are initialized close to the identity function, so training begins from behavior indistinguishable from the unmodified model and the adapter only has to learn a perturbation, not a function from scratch; this is what keeps training stable even though every adapter is initialized independently and nothing coordinates them across layers. On the GLUE benchmark, Houlsby et al. report performance within 0.4% of full fine-tuning while adding only about 3.6% additional parameters per task, a result achieved by touching every transformer block rather than concentrating the trainable budget in a few strategic locations, which is part of why the added parameter count, while small in relative terms, is not negligible in absolute ones once a network has dozens of blocks.

Training one adapter per task and then wanting several of them to work together exposes a second problem that adapters, uniquely among the families in this survey, have a mature answer to. AdapterFusion (Pfeiffer et al., 2021) freezes a set of independently trained task adapters and learns a new attention layer on top of them that decides, per input, how to combine their outputs, achieving composition without further gradient updates to the adapters themselves and therefore without the interference that comes from continuing to fine-tune a shared representation across tasks. Compacter (Mahabadi et al., 2021) attacks a different axis of the same design: it replaces an adapter's dense projection matrices with a sum of Kronecker products between weights shared across every layer and small low-rank weights specific to each one, cutting the parameter count by roughly another order of magnitude without changing what the module computes in kind.

None of this changes what an adapter is at inference time: an extra piece of the forward pass, executed on every request, for as long as the model is deployed with it attached. Because the bottleneck's nonlinearity sits between the down- and up-projections, the module cannot be algebraically folded into the frozen weight matrix the way a purely linear update can; there is no way to precompute a new weight matrix that behaves like "model plus adapter" and then discard the adapter. Reaching for adapters specifically, rather than for the low-rank methods described next, makes sense when the deployment genuinely needs several task modules alive behind one frozen backbone and a principled way to combine them, the way AdapterFusion provides; for a single task with no composition requirement, that architectural cost buys little that a mergeable low-rank update does not already give more cheaply.

4 Low-Rank Reparameterization: LoRA and Its Variants

LoRA (Hu et al., 2021) leaves a pretrained weight matrix frozen and adds a low-rank update computed as the product of two much smaller matrices, with the rank set far below the dimensions of the matrix being adapted; one of the two factors is initialized at zero, so the update contributes nothing at the start of training and the model's behavior only drifts as far as the data pushes it. Because trainable-parameter count scales with the rank times the matrix's dimensions rather than with the matrix's full size, the reduction can be extreme: against GPT-3 175B fine-tuned with Adam, the authors report a 10,000-fold cut in trainable parameters and a threefold cut in GPU memory. What makes LoRA more than an efficient way to train, though, is what happens after training: the low-rank update is added directly into the frozen matrix in weight space, so it can be merged permanently: compute the sum once, discard the two low-rank factors, and the resulting weight matrix behaves exactly like an ordinarily fine-tuned one at inference, with no adapter module, no extra sequence length, and no latency cost beyond what the unmodified base model already pays.

Four descendants push on different parts of that design. QLoRA (Dettmers et al., 2023) does not change what the low-rank update does; it changes what the frozen backbone is stored as, keeping it in a 4-bit quantized format and training LoRA adapters on top of it in higher precision, combined with a second round of quantizing the quantization constants themselves and an optimizer that pages memory to the CPU rather than spiking on gradient-checkpointing peaks. The result is the paper's headline claim: a 65-billion-parameter model that needed more than 780 gigabytes to fine-tune fully now fits, adapters and all, on a single 48-gigabyte GPU, without giving up the task performance a full 16-bit fine-tune would reach. AdaLoRA (Zhang et al., 2023) instead questions LoRA's assumption that every adapted matrix deserves the same fixed rank: it parameterizes the update in a singular-value-decomposition form, scores each singular value's importance as training proceeds, and prunes the least important ones, so the rank budget migrates toward the weight matrices that need it and away from the ones that do not. DoRA (Liu et al., 2024) decomposes each pretrained weight into a magnitude and a direction, leaves the magnitude free to move and restricts only the direction to a LoRA-style low-rank update, a change the authors report closes more of the remaining gap to full fine-tuning than vanilla LoRA does at a matched rank, an effect that grows sharper the lower the rank is pushed. VeRA (Kopiczko et al., 2024) goes the other way on parameter count: it shares one frozen, randomly initialized pair of low-rank projections across every adapted layer in the network and trains only small per-layer scaling vectors on top of that shared pair, reporting roughly a further order-of-magnitude reduction in trainable parameters relative to standard LoRA at comparable task performance.

For most single-task, self-hosted fine-tuning of an open-weight model, vanilla LoRA, or DoRA specifically when the rank budget is tight enough that every point of accuracy

matters, is the reasonable default: both produce a mergeable, zero-overhead artifact with a well-understood training recipe. QLoRA is not a strictly better LoRA; it answers a different question, and the question is whether the base model fits in memory at all, not whether it is worth the added quantization complexity when it already does. AdaLoRA and VeRA are worth the extra machinery only when there is a concrete reason to care about their specific axis (a genuinely uneven importance across weight matrices, or an extreme constraint on how many parallel adapters need to fit in memory at once), rather than as something to reach for by default.

5 Prompt- and Prefix-Based Tuning

Prompt tuning (Lester et al., 2021) does not touch the model's weights at all: it prepends a short sequence of learnable embedding vectors to the input and trains only those vectors, leaving every pretrained parameter, including the ordinary token embedding table, exactly as it was. The central finding is not the mechanism itself but its dependence on scale: as the underlying model grows into the tens of billions of parameters, the gap between prompt tuning and full model tuning closes, to the point that at the largest scales tested the two become difficult to tell apart on standard benchmarks. That dependence cuts the other way at smaller scales: a handful of input-level vectors is often not enough leverage to steer a smaller frozen model into genuinely different task behavior, which is why prefix tuning (Li and Liang, 2021) extends the same idea into every transformer layer, prepending learnable key and value vectors directly to each layer's attention computation rather than only at the input. That gives the frozen network far more surface to be steered through, and the authors report performance comparable to full fine-tuning on table-to-text generation and summarization while tuning roughly 0.1% of parameters, with an actual advantage over fine-tuning in the low-data regime.

P-Tuning v2 (Liu et al., 2021) is best read as confirming, rather than contradicting, the scale story: it takes the same deep, per-layer prompt architecture as prefix tuning and applies it specifically to close the gap prompt tuning still has at small and medium model sizes and on harder natural-language-understanding tasks, where the original input-only formulation had underperformed. Across a wide range of model sizes, the deep formulation matches fine-tuning while touching between roughly 0.1% and 3% of parameters, evidence that the scale-dependence problem was a property of where the prompt was attached, not an intrinsic limit of prompting the model at all.

None of this is free at inference, just not-free in a different currency than an adapter is. A soft prompt occupies real positions in the sequence the model processes, and a learned prefix occupies real entries in the key/value cache at every layer, for as long as the request is being served: context budget and cache memory spent on every call, proportional to how many virtual tokens the method uses. That cost profile is exactly why this family earns its keep in one specific situation: serving behind an interface where the weights

genuinely cannot be touched, such as a closed model accessed only through an API, or a setting where switching between many tasks by swapping a short vector matters more than shaving inference cost. Once the weights are available to fine-tune directly, as they are for any open-weight model, LoRA gives a mergeable result for less continuing cost, and prompt or prefix tuning stops being the first thing to reach for.

6 Selective and Sparse Fine-Tuning

The most restrictive member of this family trains nothing but bias terms. BitFit (Ben Zaken et al., 2021) freezes every weight matrix in the network and updates only the additive bias parameters in its linear layers and layer norms (typically well under a tenth of a percent of the total parameter count) and reports this alone is competitive with full fine-tuning on GLUE, particularly when the training set is small, where it can even outperform updating every weight. Diff pruning (Guo et al., 2021) relaxes that restriction while keeping the same spirit: rather than fixing in advance which parameters may change, it learns a task-specific diff vector added on top of the full pretrained parameters and sparsifies that vector during training with a differentiable relaxation of the L0 penalty, so which roughly half a percent of parameters actually move is learned per task rather than pinned to the bias terms alone; the authors report this matches fully fine-tuned baselines on GLUE at that sparsity level. $(IA)^3$ (Liu et al., 2022) sits close to both: it learns three vectors per layer that rescale the attention keys, attention values, and feed-forward activations elementwise, adding no new module and no new sequence positions. Used inside the T-Few recipe on top of the T0 model, it is reported to outperform few-shot in-context learning on held-out tasks while costing a fraction of the inference compute that carrying in-context examples in every prompt requires.

All three share the mergeability property usually credited to LoRA alone, though by a different route in each case. BitFit's trained bias and diff pruning's sparse diff vector are literally additive replacements for a piece of the frozen parameters (the same shape as the thing they modify, added in), so both can be folded into the base model permanently with no residual module left behind. $(IA)^3$'s rescaling is multiplicative rather than additive, but a per-channel rescale can still be absorbed into an adjacent weight matrix once and for all, so it keeps the same zero-overhead property by an algebraically different argument.

This family's reputation as a lightweight also-ran undersells BitFit specifically. When labeled data is scarce and the task is not too far from what the model already saw in pretraining, training a few thousand bias parameters is a same-day experiment, not a fallback to try after everything else has failed, and the fact that it sometimes beats full fine-tuning in exactly that regime is a stronger result than its parameter count suggests. Diff pruning is harder to recommend on the same terms today: its sparsification machinery adds real training-time cost for a result that LoRA now matches or beats with

a simpler recipe, and its lasting contribution is closer to an existence proof, that well under one percent of a weight-space edit can suffice, than a method worth reaching for in production.

7 Cross-Family Comparison

Every method described so far can be placed on three axes that matter to whichever engineer has to actually ship it: how many parameters it trains, what it costs the moment a request hits the deployed model, and how well it composes when more than one task-specific version needs to exist behind the same backbone.

Method	Typical trainable parameters	Cost at inference	Multi-task composition
Adapters (Houlsby)	~2-4%	permanent module in the forward pass	AdapterFusion: mature, non-destructive
Compacter	~0.05-0.15%	permanent module in the forward pass	via adapter-style fusion
LoRA / QLoRA / AdaLoRA / DoRA	~0.1-0.5%	none, once merged	merge or hot-swap; no principled fusion
VeRA	~0.01-0.05%	none, once merged	merge or hot-swap
Prompt tuning	<0.01%	permanent extra sequence positions	swap a short vector
Prefix tuning / P-Tuning v2	~0.1-3%	permanent KV-cache entries, every layer	swap a prefix set
BitFit	<0.1%	none, once applied	little published work
Diff pruning	~0.5%	none, once applied	little published work
$(IA)^3$	<0.01%	none, once folded into weights	little published work

Read by the trainable-parameter column alone, the table looks like a simple ranking, from prompt tuning and $(IA)^3$ at the bottom through adapters at the top. Read by the inference-cost column, the same nine rows sort into only two groups, and the boundary between them runs diagonally through the taxonomy built across the last four sections

rather than along it: the LoRA family and the selective/sparse family both fold their trained delta back into the frozen weights and cost nothing extra once that happens, while adapters and prompt-based methods both keep something running in the forward pass or occupying the sequence for as long as the model is deployed. Mechanism and deployment cost, in other words, are answers to two different questions, and a taxonomy organized around the first tells a reader nothing reliable about the second.

The gap between the two columns is largest exactly where it is least expected. $(IA)^3$ trains on the order of a few hundred thousand parameters per model, fewer than a typical rank-8 LoRA update on the same network, and shares LoRA's zero-overhead property once its rescaling vectors are folded in; a Houlsby-style adapter at a modest bottleneck width can train more parameters than that same rank-8 LoRA update and still pay a forward-pass cost on every request, forever. "More parameter-efficient" and "cheaper to serve" are correlated across the field in general, which is presumably why the two get conflated, but they are not the same ordering method by method, and a team optimizing for serving cost that reads only the parameter-count column is optimizing for the wrong number.

Composition is the one column where the families do not scramble: adapters keep the most developed answer, because AdapterFusion was built specifically to combine several of them without further training. LoRA modules can be merged or swapped at inference, but nothing in the LoRA literature offers AdapterFusion's principled, learned way of deciding how much of each to use for a given input, a gap the next section returns to directly.

8 Open Problems

Two questions in this literature remain genuinely open, in the sense that existing work does not resolve them so much as work around them, and both are worth arguing in full rather than filed alongside secondary concerns like hyperparameter sensitivity or extension to architectures beyond the transformer, which are real but comparatively well understood by comparison.

The first is that nobody has explained why these particular mechanisms work as well as they do, as opposed to merely having shown that a mechanism exists which works. Aghajanyan et al. (2021) demonstrated that pretrained language models have a low intrinsic dimension for fine-tuning: a small subspace of the full parameter space is nearly as effective to update as the whole thing, and that subspace shrinks further the longer a model has been pretrained. That result explains why fine-tuning in some low-dimensional form should work at all; it does not explain why a low-rank edit, a sparse edit, and an input-space edit, all nominally exploiting the same low-dimensional structure, land in noticeably different empirical regimes depending on model scale and task. He et al. (2022) took the unification a step further by showing that adapters, prefix tuning, and

LoRA can all be written as instances of one general framework for injecting a learned change into the frozen computation — a genuine result, and useful for generating new hybrid methods, but descriptive rather than explanatory: a framework flexible enough to express every method in this survey equally well cannot, by construction, be the reason any one of them outperforms another. This gap is not merely academic, because the empirical trend cuts against the taxonomy this survey has spent four sections building: as base models get larger, the reported performance gaps between families tend to narrow, and it is not clear whether that narrowing means the mechanism differences genuinely stop mattering at scale, or whether it means the benchmarks used to detect them have already saturated and are no longer sensitive enough to see where the differences persist. Nothing in the current literature adjudicates between those two readings, and this survey does not either: the honest position is that the field has not earned a preference between them yet.

The second problem is what happens when more than one of these adaptations needs to exist on the same backbone at once. LoraHub (Huang et al., 2023) shows that a handful of already-trained LoRA modules can be linearly combined, with the combination weights fit from a few examples of a new task, to generalize to that task without any gradient-based retraining. TIES-Merging (Yadav et al., 2023) shows that two specific sources of damage during merging, small likely-noisy parameter changes and outright sign disagreements between task vectors on the same parameter, can be identified and corrected before averaging, reducing the interference that naive merging produces. Both are real, validated recipes, and both are recipes: they are tested after the fact on the task combinations the authors happened to choose, and neither offers a test that can be run on two candidate adapters beforehand to say whether merging them will help or actively hurt. Practitioners today decide whether to merge two adaptations by trying it and checking, which is exactly the position full fine-tuning research was in before PEFT itself supplied a theory of what could be thrown away safely.

Put next to each other, the two problems are one problem wearing different clothes. The field has strong empirical unification in both places (a mechanism-level framework flexible enough to describe every method here, and a merging toolkit that resolves most of the conflicts it is tested against) and weak explanatory unification in both places: no account of why a particular mechanism wins where it does, no predictive test of which merges will conflict before trying them. That gap between describing after the fact and predicting in advance, not a missing method or a missing benchmark, is the open problem this survey leaves standing.

9 Conclusion

Set next to each other, the four families reduce to a short decision procedure rather than a leaderboard. Default to LoRA, or to DoRA specifically when the rank budget is

tight enough that every point of accuracy matters, for self-hosted fine-tuning of an open-weight model where a mergeable, zero-overhead result is the goal. Reach for adapters when several task-specific versions of one model need to coexist and be combined without retraining, since AdapterFusion's composition story has no real equivalent elsewhere in this survey. Reach for prompt or prefix tuning only when the backbone genuinely cannot be touched, which today mostly means access through a closed API rather than an open-weight model on hand. Try a selective method, BitFit above all, before anything heavier, when labeled data is scarce.

The four-family taxonomy that organized the last six sections was a teaching device, not a claim about what determines cost or behavior; its seams are exactly where the two arguments above land: the inference-cost axis that cuts diagonally through it, and the absence of any theory predicting which mechanism wins where, or which merged adaptations will conflict. Those seams, not another variant to catalogue, are where this literature still has real work to do.

10 References

1. Aghajanyan, A., Gupta, S., & Zettlemoyer, L. (2021). Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. *ACL-IJCNLP 2021*. arXiv:2012.13255.
2. Ben Zaken, E., Ravfogel, S., & Goldberg, Y. (2021). BitFit: Simple Parameter-Efficient Fine-Tuning for Transformer-Based Masked Language-Models. arXiv:2106.10199.
3. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314.
4. Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W., Yi, J., Zhao, W., Wang, X., Liu, Z., Zheng, H.-T., Chen, J., Liu, Y., Tang, J., Li, J., & Sun, M. (2022). Delta Tuning: A Comprehensive Study of Parameter Efficient Methods for Pre-trained Language Models. arXiv:2203.06904.
5. Guo, D., Rush, A. M., & Kim, Y. (2021). Parameter-Efficient Transfer Learning with Diff Pruning. *ACL-IJCNLP 2021*. arXiv:2012.07463.
6. He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2022). Towards a Unified View of Parameter-Efficient Transfer Learning. *ICLR 2022*. arXiv:2110.04366.
7. Houlsby, N., Giurgiu, A., Jastrzębski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-Efficient Transfer Learning for NLP. *ICML 2019*. arXiv:1902.00751.
8. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
9. Huang, C., Liu, Q., Lin, B. Y., Pang, T., Du, C., & Lin, M. (2023). LoraHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition. arXiv:2307.13269.
10. Kopiczko, D. J., Blankevoort, T., & Asano, Y. M. (2024). VeRA: Vector-Based Random Matrix Adaptation. *ICLR 2024*. arXiv:2310.11454.

11. Lester, B., Al-Rfou, R., & Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. *EMNLP 2021*. arXiv:2104.08691.
12. Li, X. L., & Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. *ACL 2021*. arXiv:2101.00190.
13. Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., & Raffel, C. (2022). Few-Shot Parameter-Efficient Fine-Tuning Is Better and Cheaper than In-Context Learning. *NeurIPS 2022*. arXiv:2205.05638.
14. Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. *ICML 2024*. arXiv:2402.09353.
15. Liu, X., Ji, K., Fu, Y., Tam, W. L., Du, Z., Yang, Z., & Tang, J. (2021). P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-Tuning Universally Across Scales and Tasks. arXiv:2110.07602.
16. Mahabadi, R. K., Henderson, J., & Ruder, S. (2021). Compacter: Efficient Low-Rank Hypercomplex Adapter Layers. *NeurIPS 2021*. arXiv:2106.04647.
17. Pfeiffer, J., Kamath, A., Rücklé, A., Cho, K., & Gurevych, I. (2021). Adapter-Fusion: Non-Destructive Task Composition for Transfer Learning. *EACL 2021*. arXiv:2005.00247.
18. Yadav, P., Tam, D., Choshen, L., Raffel, C., & Bansal, M. (2023). TIES-Merging: Resolving Interference When Merging Models. *NeurIPS 2023*. arXiv:2306.01708.
19. Zhang, Q., Chen, M., Bukharin, A., Karampatziakis, N., He, P., Cheng, Y., Chen, W., & Zhao, T. (2023). AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *ICLR 2023*. arXiv:2303.10512.